

CSE 3131

Compiler Design

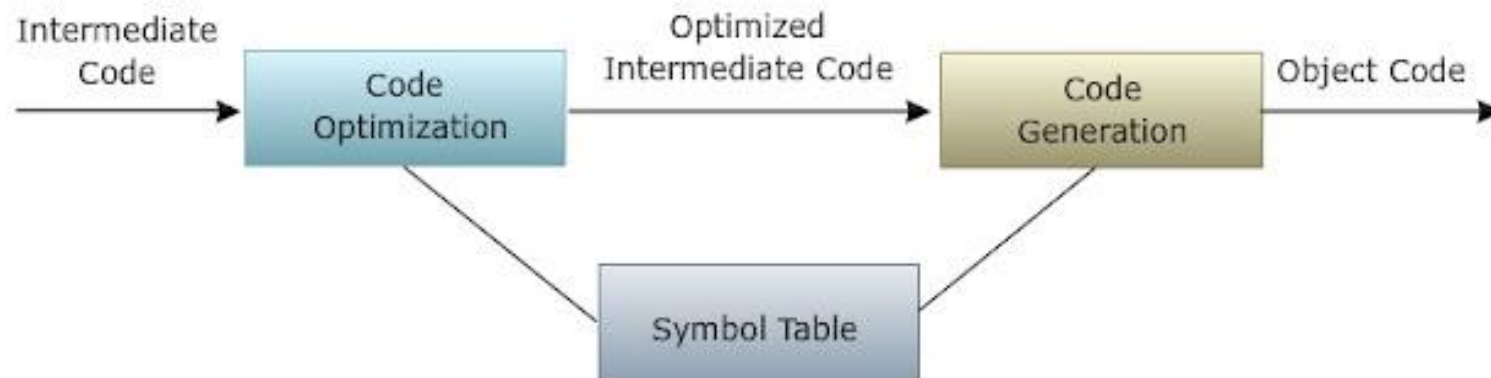
Md. Emdadul Haque

Lecturer

Department of Computer Science & Engineering
North Bengal International University

Code Optimization

- The code optimization in the synthesis phase is a program transformation technique, which tries to improve the intermediate code by making it consume fewer resources (i.e. CPU, Memory) so that faster-running machine code will result.



Reasons Why We Need Code Optimization

- 1. Cleaner Code Base**
- 2. Higher Consistency**
- 3. Better Code Readability**
- 4. More Efficient**
- 5. More Straightforward Debugging**
- 6. Improved Workflow**
- 7. Easier Code Maintenance**
- 8. Smaller Technical Debt**

What is code optimization? Why we need it?

Code Optimization Techniques

Broadly there are two techniques to optimize the code

Machine Independent Tech.

- Loop optimization*
(Code motion ,loop fusion, loop unrolling)
- Constant propagation
- Constant folding

Machine Dependent Tech.

- Peephole optimization*
- Instruction level parallelism
- Data level Parallelism

Loop Optimization

Loop Optimization is the process of increasing execution speed and reducing the overheads associated with loops. It plays an important role in improving cache performance and making effective use of parallel processing capabilities. Most execution time of a scientific program is spent on loops.

Loop Optimization is a machine independent optimization.

1. Code motion(frequency reduction)

Suppose a code:

```
a=100;
while(a>0)
{
X=Y+Z;
If ( a%x == 0 )
Printf(“%d”,a);
i--;
}
```

Code motion(frequency reduction)

Optimized.

```
a=100;
X=Y+Z;
while(a>0)
{
If ( a%x == 0 )
Printf(“%d”,a);
i--;
}
```

What is loop optimization? Describe with some suitable example.

Loop Optimization (Contd.)

2. Loop fusion

Suppose a code:

```
Int i, a[100], b[100];  
For (i=0; i<100; i++)  
    a[i] =1;  
For (i=0; i<100; i++)  
    b[i]=2;
```

2. Loop fusion

Optimized

```
Int i, a[100], b[100];  
For (i=0; i<100; i++)  
    a[i] =1;  
    b[i]=2;
```

What is loop optimization? Describe with some suitable example.

Loop Optimization (Contd.)

3. Loop unrolling

Suppose a code:

```
For (i=0; i<4; i++)  
    Printf ("north_bengal");
```

3. Loop unrolling

Optimized

```
Printf ("north_bengal");  
Printf ("north_bengal");  
Printf ("north_bengal");  
Printf ("north_bengal");
```

What is loop optimization? Describe with some suitable example.

Peephole Optimization

Peephole optimization is a type of Code optimization performed on a small part of the code. It is performed on the very small set of instructions in a segment of code.

- ❖ The small set of instructions or small part of code on which peephole optimization is performed is known as peephole or window.
- It basically works on the theory of replacement in which a part of code is replaced by shorter and faster code without change in output.
- Peephole is the machine dependent optimization.

Objectives of Peephole Optimization:

- To improve performance
- To reduce memory footprint
- To reduce code size

What is peephole optimization? Write down the objectives of peephole optimization.

Peephole Optimization Techniques

1. Redundant load and store elimination:

In this technique the redundancy is eliminated.

- Initial code:

```
y = x + 5;
```

```
i = y;
```

```
z = i;
```

```
w = z * 3;
```

- Optimized code:

```
y = x + 5;
```

```
i = y;
```

```
w = y * 3;
```

What is peephole optimization? Describe with some suitable example.

Peephole Optimization Techniques (contd.)

1. Redundant load and store elimination:

```
int add_ten(int x)
{
    int y, z;
    y = 10;
    z = x + y;
    return z;
}
```

```
int add_ten(int x)
{
    int y;
    y = 10;
    y = x + y;
    return y;
}
```

```
int add_ten(int x)
{
    int y = 10;
    return x + y;
}
```

```
int add_ten(int x)
{
    return x + 10;
}
```

What is peephole optimization? Describe with some suitable example.

Peephole Optimization Techniques (contd.)

2. Strength Reduction:

The operators that consume higher execution time are replaced by the operators consuming less execution time.

- Initial code:

$y = x * 2;$

- Optimized code:

$y = x + x;$ or $y = x \ll 1;$

- Initial code:

$y = x / 2;$

- Optimized code:

$y = x \gg 1;$

What is peephole optimization? Describe with some suitable example.

Peephole Optimization Techniques (contd.)

3. Constant folding:

Constant folding is the process of recognizing and evaluating constant expressions at compile time rather than computing them at runtime.

The code that can be simplified by user itself, is simplified.

- Initial code:
- $x = 2 * 3;$
- Optimized code:
- $x = 6;$

What is peephole optimization? Describe with some suitable example.

Peephole Optimization Techniques (contd.)

4. Simplify algebraic expressions

a=a+0;

a=a*1; **NO NEED**

a=a/1;

a=a-0;

5. Replace slower instructions with faster

Add 1,R → INC R
Sub 1,R → DEC R

a load X

a load X

MUL

a load X

dup

MUL

What is peephole optimization? Describe with some suitable example.

Peephole Optimization Techniques (contd.)

6. Dead Code Elimination

Dead Code Elimination is an optimization that removes code which does not affect the program results. Removing such code has several benefits: it shrinks program size and it allows the running program to avoid executing irrelevant operations, which reduces its running time.

```
int dead (void)
{
    int a=10;
    int b=20;
    int c;
    c= a*10;
    return c;
    b=30;
    b=b*10;
    return 0;
}
```

What is peephole optimization? Describe with some suitable example.

Basic Block

- Basic block contains a sequence of statement. The flow of control enters at the beginning of the statement and leave at the end without any halt (except may be the last instruction of the block).
- **How to find leader in basic block? (Algorithm)**
 1. First statement is always a leader
 2. Address of conditional, unconditional goto are leaders
 3. Next line of conditional and unconditional goto are leaders.
 4. Make basic block from leader to line before next leader.
- ** First line of basic block is leader.

Write down the rules associated with the construction of basic block?

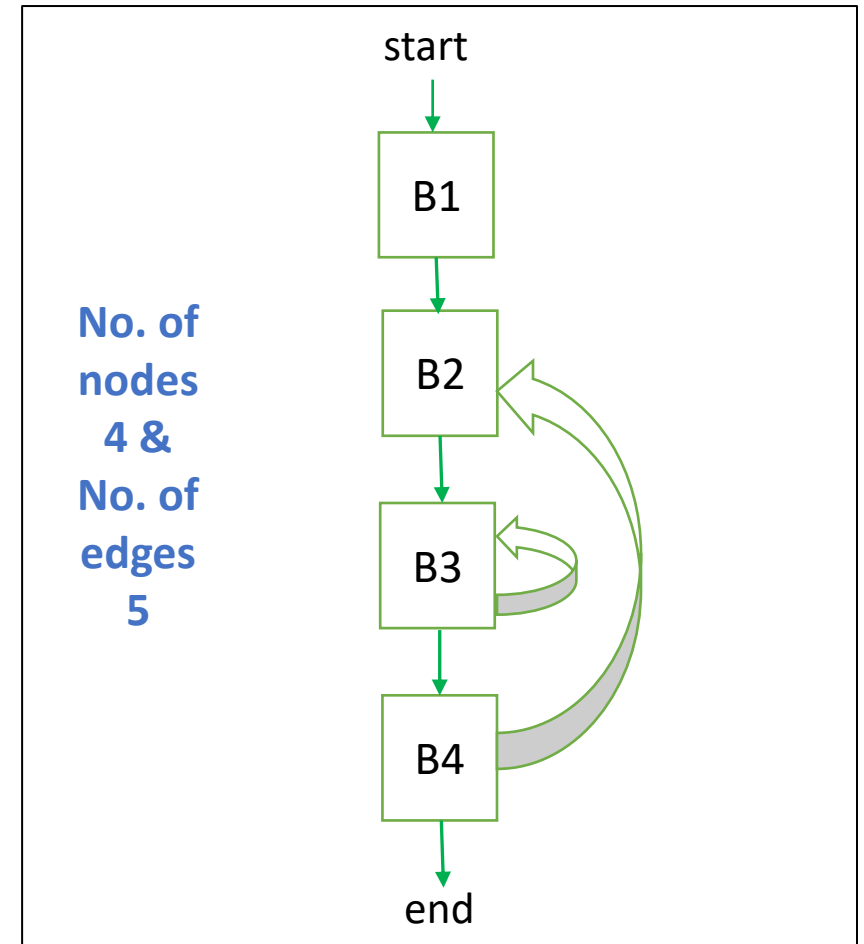
Basic Block

- Consider the intermediate code:

```
1. i=1; Leader
2. J=1; Leader
3. t1=5*I Leader
4. t2=t1+j
5. t3=4*t2
6. t4=t3
7. a[t4]=-1
8. j=j+1
9. if j<=5 goto (3)
10. i=i+1 Leader
11. if i<5 goto (2)
```

```
1. i=1; Leader
2. J=1; Leader
3. t1=5*I Leader
4. t2=t1+j
5. t3=4*t2
6. t4=t3
7. a[t4]=-1
8. j=j+1
9. if j<=5 goto (3)
10. i=i+1 Leader
11. if i<5 goto (2)
```

Control flow graph



Find the no. of nodes and edges in control flow graph from the given example?

Basic Block & Control flow graph

- A control flow graph depicts how the program control is being passed among the blocks. It is useful in the loop optimization.

B1

```
w = 0;  
x = x + y;  
y = 0;  
if( x > z)
```

B2

```
y = x;  
x++;
```

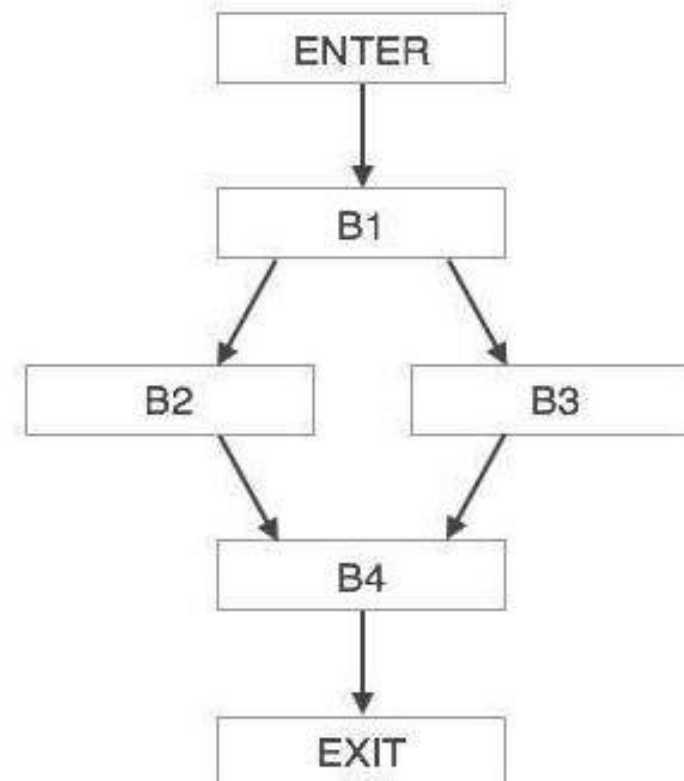
B3

```
y = z;  
z++;
```

B4

```
w = x + z;
```

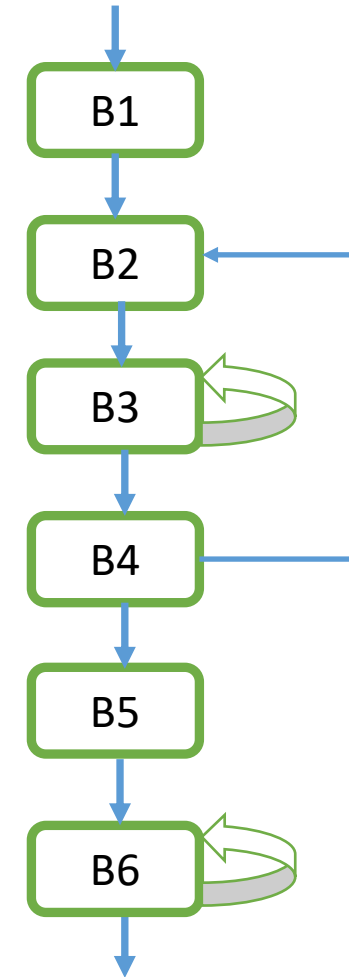
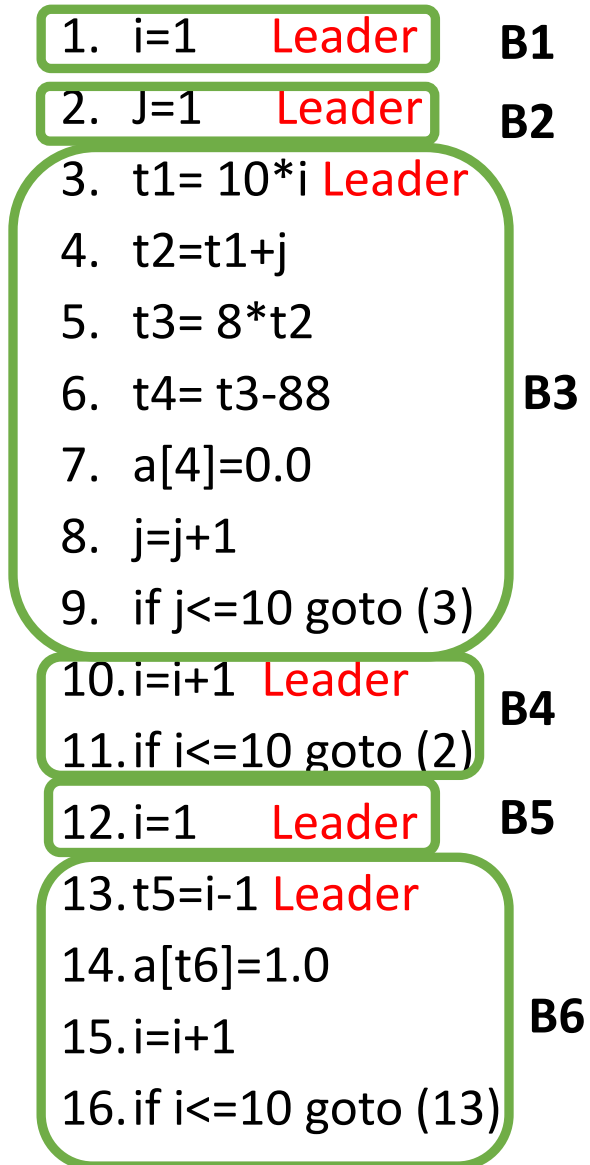
Basic Blocks



Flow Graph

Find the no. of nodes and edges in control flow graph from the given example?

Basic Block & Control flow graph



Find the no. of nodes and edges in control flow graph from the given example?

THANKS